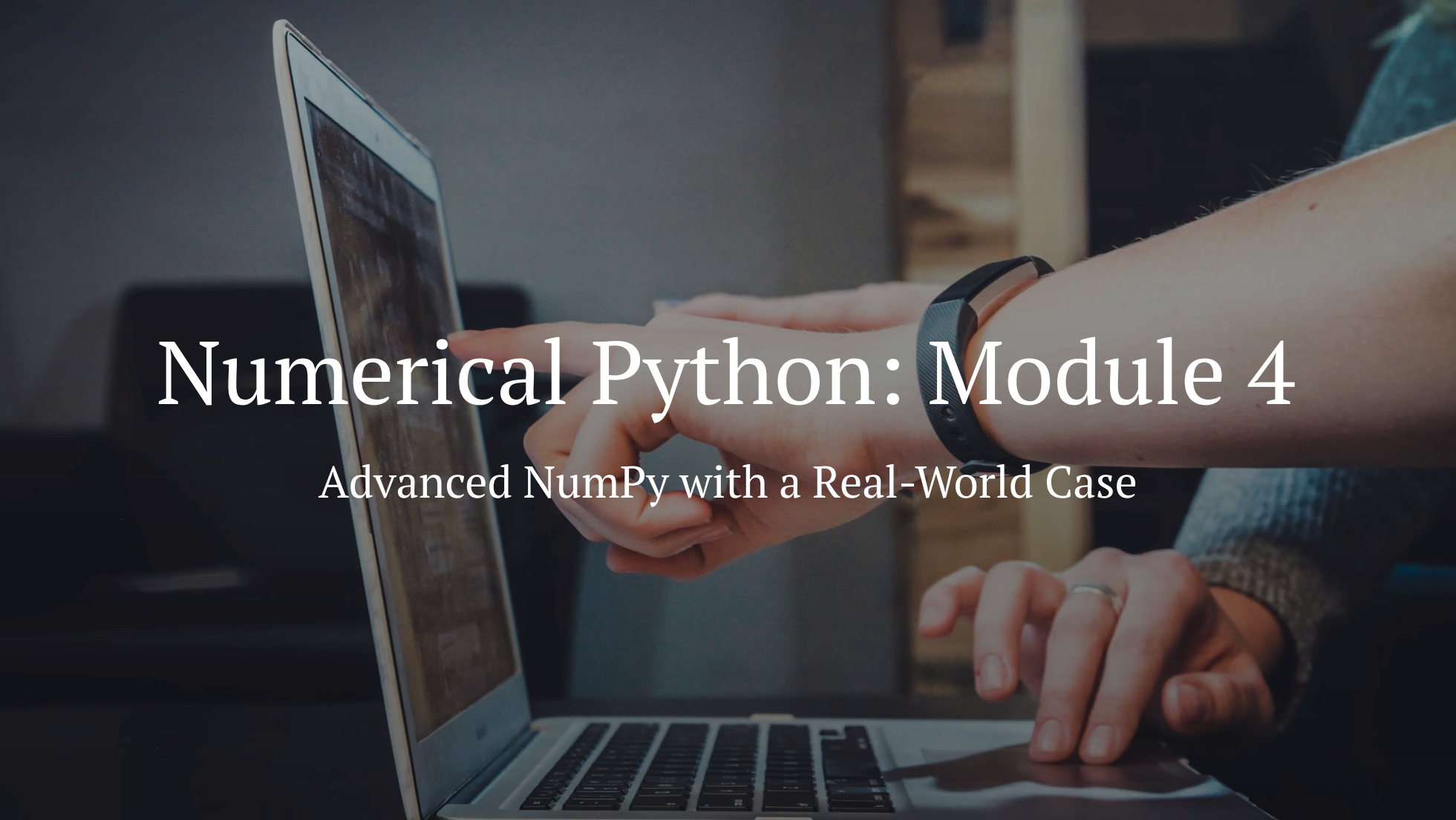

Numerical Python: Module 4

Advanced NumPy with a Real-World Case

1. Goal of This Lecture

In the basics lecture, you learned how to create arrays and do simple operations.

In this lecture, we move to the next level:

1. Work with **multi-dimensional real data**
2. Use **broadcasting** instead of manual loops
3. Filter data with **boolean masks**
4. Summarize data by **axis**
5. Solve a small **real-world analytics problem**

2. Real-World Problem

Smart Building Energy Analysis

Imagine a company manages a smart office building.

Every hour, sensors record electricity usage (kWh) for 5 rooms across 7 days.

We want to answer questions like:

- Which room uses the most energy?
- Which day had unusually high usage?
- Which rooms exceed a daily energy budget?
- How can we normalize data to compare patterns fairly?

3. Our Dataset

```python
import numpy as np

# rows = days, columns = rooms
energy = np.array([
    [30, 45, 22, 60, 35],
    [32, 47, 21, 58, 36],
    [31, 50, 25, 65, 38],
    [29, 43, 20, 55, 34],
    [35, 52, 26, 70, 40],
    [40, 60, 30, 80, 45],
    [38, 58, 28, 78, 44]
])

print(energy.shape)   # (7, 5)
```

This is a small dataset, but the exact same approach works for thousands of days and hundreds of sensors.

4. Think in Axes

For a 2D NumPy array:

- `axis=0` means **down the rows** -> summarize each column
- `axis=1` means **across the columns** -> summarize each row

```
# total usage per room
print(energy.sum(axis=0))

# total usage per day
print(energy.sum(axis=1))

# average usage per room
print(energy.mean(axis=0))
```

5. Finding Patterns Fast

```
room_totals = energy.sum(axis=0)
day_totals = energy.sum(axis=1)

most_expensive_room = np.argmax(room_totals)
highest_usage_day = np.argmax(day_totals)

print(room_totals)
print(day_totals)
print(most_expensive_room)
print(highest_usage_day)
```

Why this matters

`argmax()` is common in analytics, recommendation systems, and monitoring dashboards.

6. Boolean Masking

We often need to filter values based on conditions.

```
# Find all readings above 50 kWh
high_usage = energy > 50
print(high_usage)

# Extract only those values
print(energy[high_usage])

# Count how many high-usage readings exist
print(np.sum(high_usage))
```

This is much cleaner than writing nested `for` loops with `if` statements.

7. Daily Budget Check

Suppose each room should stay under this daily target:

```
budget = np.array([35, 50, 25, 65, 40])
```

Now compare every day against the room budget:

```
over_budget = energy > budget
print(over_budget)
```

Key idea

This works because of **broadcasting**.

NumPy automatically stretches `budget` across each row.

8. Broadcasting Explained

```
energy.shape   # (7, 5)
budget.shape   # (5,)
```

NumPy treats `budget` like this:

```
[
  [35, 50, 25, 65, 40],
  [35, 50, 25, 65, 40],
  ...
]
```

without actually copying the data.

Result

Fast comparisons, less memory, and cleaner code.

9. Where Did We Exceed the Budget?

```python
over_budget = energy > budget

# Count over-budget days per room
print(over_budget.sum(axis=0))

# Count over-budget rooms per day
print(over_budget.sum(axis=1))

# Replace normal values with 0, keep only over-budget readings
flagged = np.where(over_budget, energy, 0)
print(flagged)
```

10. Normalize for Fair Comparison

Room 4 always uses more power because it may contain heavy equipment.

To compare patterns fairly, normalize each room by its average:

```
room_means = energy.mean(axis=0)
normalized = energy / room_means

print(room_means)
print(normalized.round(2))
```

Use case

Normalization is common in finance, health data, and machine learning.

11. Detect Unusual Days

One simple method: use a z-score style approach.

```
day_totals = energy.sum(axis=1)

mean_total = day_totals.mean()
std_total = day_totals.std()

z_scores = (day_totals - mean_total) / std_total
print(z_scores.round(2))
```

```
unusual_days = np.abs(z_scores) > 1.2
print(unusual_days)
```

This is a simple anomaly detection idea used in operations and monitoring systems.

12. Real Output Interpretation

From this dataset we can conclude:

1. Room 4 is the biggest energy consumer
2. The weekend-like days at the end have the highest total usage
3. Some rooms repeatedly exceed their budget
4. Normalization helps us compare patterns, not just raw size

This is the real value of NumPy

It helps transform raw numbers into decisions.

13. Loop Version vs NumPy Version

```python
# Slow, manual style
totals = []
for col in range(energy.shape[1]):
    total = 0
    for row in range(energy.shape[0]):
        total += energy[row, col]
    totals.append(total)
```

```python
# Fast, NumPy style
totals = energy.sum(axis=0)
```

Better NumPy code is usually

- shorter
- easier to read
- faster on large data

14. Mini Exercise 1

Using the `energy` array:

1. Find the average energy usage for each day
2. Find the index of the room with the lowest total usage
3. Count how many readings are below `30`

```
# Write your code here
```

15. Mini Exercise 2

Using the room budget:

```
budget = np.array([35, 50, 25, 65, 40])
```

Do the following:

1. Create a boolean mask for values over budget
2. Count how many times each room exceeded budget
3. Print only the over-budget values

16. Mini Exercise 3

Create a new array called `discounted_energy` where:

- values above `60` are reduced by 10%
- other values stay the same

Hint:

```
np.where(condition, value_if_true, value_if_false)
```

17. Challenge Exercise

Office Expansion Decision

A manager wants to renovate one room to reduce long-term energy cost.

Your task:

1. Find the room with the highest **total** energy usage
2. Find the room with the highest **average normalized** usage
3. Decide whether the same room should be prioritized
4. Explain your reasoning in 2-3 sentences

This mirrors real analytics work

The goal is not just calculation. The goal is making a justified decision.

18. Exercise Solutions

```
1
```

19. More Solution Ideas

```
1
```

20. Summary

In advanced NumPy work, the core ideas are:

- **Axis-based thinking**
- **Broadcasting**
- **Boolean masking**
- **Vectorized transformation**
- **Turning data into decisions**

If you can do these well, you are using NumPy the way data professionals do.

End

Reference Sources:

- NumPy Quickstart Tutorial
- NumPy Broadcasting
- NumPy `where`